COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S1
DATE PERFORMED:	July 28, 2026	**DATE SUBMITTED:**	August 1, 2026
NAME:	Carl Andrei D. Torres	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4 **Seatwork 4.1: Array-Based vs STL Stack Implementations**

1. PROCEDURE OUTPUT

ILO A

```cpp
#include <iostream>
#include <stack> //calling stack from the STL


int main(){
    //create an object for the stack
    std::stack<int> newStack;


    std::cout<<"Stack Empty?" << newStack.empty() << std::endl;

    //push to the stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout<<"Stack Empty?" << newStack.empty() << std::endl;
    std ::cout<<"The Size of the Stack: "<<newStack.size()<<std::endl;
    std::cout<<"The Top of the Element is: "<<newStack.top()<<std::endl;

    //popping the stack
    std::cout<<"Removing Elements from the Stack: \n";
    newStack.pop();
    std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;

    return 0;
}
```

Main Driver Code

```
Stack Empty?1
Stack Empty?0
The Size of the Stack: 3
The Top of the Element is: 15
Removing Elements from the Stack:
The top of the element is: 8
The top of the element is: 3
Stack Empty? 1
```

OUTPUT

ILO B

```cpp
#include <iostream>

//Static Array
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;

void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void displayStack();

int main(){
    int choice;
    std::cout<< "Enter the number of max elements for the new stack: ";
    std::cin>>i;

   while(true){
        std::cout<<"Stack Operations: "<<std::endl;
        std::cout<<" 1. PUSH 2. POP 3. TOP 4. isEMPTY 5. displayStack"<<std::endl;
        std::cin >> choice;
        switch (choice) {
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<< isEmpty()<<std::endl;
            break;
            case 5: displayStack();
            break;
            default: std::cout<<"Invalid Choice. "<<std::endl;
            break;
    }
    }
```

```
    return 0;
}

bool isEmpty(){
    //create a condition to check whether the top element is equal to -1
    if (top == -1) return true;
    return false;
}

bool isFull(){
    //check if we reached the maximum capacity of the array:
    if(top > 100) return true;
    return false;
}

void push(){
    //check if full -> if yes return an error
    if (isFull()){
        std::cout<<"Stack Overflow";
        return;
    }

    std::cout << "New Value to add into the stack: "<<std::endl;
    std::cin >> newData;
    stack[++top] = newData;
}

void Top(){
    //check if the stack is empty
    if(isEmpty()){
        std::cout<<"Stack UnderFlow."<<std::endl;
    }

    std::cout<<"The Element on the Top of the Stack is " << stack[top]<<std::endl;
}
```

```
void pop(){
    //check if the stack is empty
    if(isEmpty()){
        std::cout<<"Stack UnderFlow."<<std::endl;
    }

    //display the top value
    std::cout<<"Popping: "<<stack[top] <<std::endl;
    //decrement the top value from the stack;
    top--;
}

void displayStack(){
    if(isEmpty()){
        std::cout<<"Stack UnderFlow."<<std::endl;
    }
    std::cout<<"Print All Elements from Top to Bottom: \n";
    for(int idx = top; idx >= 0; idx--){
        std::cout << stack[idx] << " , ";
    }
    std::cout << "\n";
}
```

Main Driver Code

```
Enter the number of max elements for the new stack: 3
Stack Operations:
 1. PUSH 2. POP 3. TOP 4. isEMPTY 5. displayStack
1
New Value to add into the stack:
1
Stack Operations:
 1. PUSH 2. POP 3. TOP 4. isEMPTY 5. displayStack
1
New Value to add into the stack:
2
Stack Operations:
 1. PUSH 2. POP 3. TOP 4. isEMPTY 5. displayStack
1
New Value to add into the stack:
3
Stack Operations:
 1. PUSH 2. POP 3. TOP 4. isEMPTY 5. displayStack
5
Print All Elements from Top to Bottom:
3 , 2 , 1 ,
Stack Operations:
 1. PUSH 2. POP 3. TOP 4. isEMPTY 5. displayStack
2
Popping: 3
Stack Operations:
 1. PUSH 2. POP 3. TOP 4. isEMPTY 5. displayStack
3
The Element on the Top of the Stack is 2
Stack Operations:
 1. PUSH 2. POP 3. TOP 4. isEMPTY 5. displayStack
4
0
```

OUTPUT

Code Analysis and Observations	In both ILO-A and ILO-B, I've learned how to implement stacks by **getting the new data by the use of PUSH** on the stack by **"newStack.push"** and the **parameters is the data you want to be added in the stack** in the **ILO-A** while in the **ILO-B** it is a **user input where you need to make a condition** if the **stack is FULL** by this code **"if (isFull()){ std::cout<<"Stack Overflow"; return; }"** . Also, **POP function** is used for **deleting a data on the stack** which you need to **search the top first** if **there's a data inside of it** then **skip it to your desired stack data** to **initiate the POP function**

	which it is **used by decrementation "top- -"**. Also, the **TOP function** is **where you display the TOP data** from the stack **which checks first if the stack is empty** before **you print the TOP data** by **" stack[top]"**. Also, the **isEmpty function** is where you **check the top element is equal to -1** where it is a **bool function which has a implementation of "if (top == -1) return true; return false"**. Also, the **displayStack function displays the stack itself by the use first the condition if the stack is empty** before **you come up to a loop that searches the TOP stack data** by this code **"for(int idx = top; idx >= 0; idx--) { std::cout << stack[idx] << " , ";}"**.

ILO B.2

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

template <typename T>
class Node{

    public:
        T data;
        Node* next;

};

//Global Pointer for the top of the stack
template<typename T>
Node<T> *head = nullptr;

//operations for the stack

//push
template<typename T>
void push(T newData){
    //create a new node ad the assign the new data
    Node<T>* newNode = new Node<T>;
    newNode -> data = newData;
    //new node points to current top, then becomes the new top
    newNode -> next = head<T>;
    head<T> = newNode;

}
//pop
template<typename T>
T pop(){
    //check if the stack is empty
    if(head<T> == nullptr){
    std :: cout << "Stack Overflow";
    }

    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
    T tempVal = temp->data;

    //move the head to next node
    head<T> = head<T> -> next;
    //free the removed node
    delete temp;
    //return the removed node
    return tempVal;
```

```cpp
    return tempVal;

}


//isEmpty
template<typename T>
bool isEmpty(){
    return head <T> == nullptr;
}

//top
template<typename T>
void Top(){
    // create a error catching:
    if (head<T> == nullptr) {
       std::cout << "stack Underflow" << std::endl;
    }
    //display the top node
    std::cout << "The Current Top of the stack is: " << head <T> -> data << std::endl;
}

//displayAll
template<typename T>
void displayAll(){
    if (isEmpty<T>()){
       std::cout<< "Stack is Empty! \n";
       return;
    }

    std::cout << "Stack (top to bottom) : ";
    Node<T>* current = head<T>;
    while (current != nullptr) {
       std::cout << current -> data;
       if (current -> next != nullptr) std::cout << " -> ";
       current = current -> next;
    }
    std::cout << std::endl;
}


#endif
```

FUNCTION CODE (STACKLIST.H)

```cpp
#include <iostream>
#include "stackList.h"

int main() {
    std::cout << "--- PUSHING 'c' 'p' 'e' ONTO THE STACK --- \n";

    push<char>('c');
    push<char>('p');
    push<char>('e');

    // Display the current top
    Top<char>();// Should display 'e'


    displayAll<char>();

    std::cout << "\n --- Popping all the elements --- \n";

    // Fixed: Loop while the stack is NOT empty
    while (!isEmpty<char>()) {
        std::cout << "Popped: " << pop<char>() << std::endl;
    }

    std::cout << "\n--- Testing the Underflow --- \n";
    pop<char>();


    return 0;
}
```

MAIN DRIVER CODE

```
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
The Current Top of the stack is: e
Stack (top to bottom) : e -> p -> c

 --- Popping all the elements ---
Popped: e
Popped: p
Popped: c

--- Testing the Underflow ---
Stack Overflow
```

OUTPUT

Conclusion and Lessons Learned	In Conclusion, I've learned how to create a stack using Arrays and Linked list which emphasized that these two have different implementation on its functions and constructor. Array-based uses fixed-size array, which makes access to elements very fast and efficient due to its direct indexing, but it requires knowing or re-adjust the maximum capacity in advance to prevent overflow if the array is full and it doesn't need to have header file. On the other hand, Linked list needs a main driver and header file. Lastly, Linked List has different implementations on its operations and constructors in the header file and main driver code because it needs to be connected from header file to main driver code to execute a specific operations.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C:

```
#include <iostream>
#include "ILOC_List.h"


int main() {
    std::cout << "---USER INPUT---\n";
    std::string expression = "{[()]}"; // Example expression to check
the right variables
    std::cout << "Enter an Expression: "; //User Input
    std::getline(std::cin, expression); //getLine to read all the
characters in the expression

    std::cout << "---Push and Pop Operations---\n";
    for (char ch : expression) {
        // Check if the character is an opening symbol, then push it
onto the stack
        if (ch == '(' || ch == '[' || ch == '{') {
            push<char>(ch);
        } else if (ch == ')' || ch == ']' || ch == '}') { //if the
character is a closing symbol, then chec against the stack
            // Pop the matching opening symbol from the stack
            if (!pop<char>(ch)) {
                // If the pop operation fails, it means the symbols are
```

```
mismatched or empty stack, it will report NOT balanced and stop the
program
                std::cout << "The expression's symbols are NOT
balanced." << std::endl;
                std::cout << "The expression is NOT VALID.\n"; //Reports
that the expression is NOT VALID
                return 0;
            }
            //If the pop operation is successful, it will display it
process
            std::cout << "Popped: " << ch << "\n";
            //shows the current top of the stack and displays all the
elements in the stack
            Top<char>();
            displayAll<char>();
        }
    }

    std::cout << "---isEmpty and Symbol Balancing Check---\n";
    if (!isEmpty<char>()) { //checks if the stack is empty at the end of
the input, if not then it will report NOT balanced or Error: stack not
empty at end of input
        std::cout << "Error: stack not empty at end of input.\n";
        displayAll<char>(); //Displays the remaining elements in the
stack
        std::cout << "The expression is NOT VALID.\n"; //Reports that
the expression is NOT VALID
    } else {
        std::cout << "Symbols are balanced!\n"; //If the stack is empty
at the end of the input, it will report that the symbols are balanced
        std::cout << "The expression is VALID.\n"; //Reports that the
expression is VALID
    }

    return 0;
}
```

MAIN DRIVER CODE

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

#include <iostream>

template <typename T>
class Node{

public:
    T data;
    Node* next;

};

//Global Pointer for the top of the stack
template<typename T>
Node<T> *head = nullptr;

//operations for the stack

//push
template<typename T>
void push(T newData) {
    // Only push the opening symbols onto the stack
    if (newData == '(' || newData == '[' || newData == '{') {
        //Allocate memory for a new node and assign the new data
        Node<T>* newNode = new Node<T>;
        // Assign the new data to the new node
        newNode->data = newData;
        // Make the new node point to the current top of the stack
        newNode->next = head<T>;
        // Update the head to point to the new node
        head<T> = newNode;
    } else {
        // Ignore non-opening symbols
        std::cout << "Ignores non-opening symbol: " << newData << "\n";
    }
}
```

```cpp
//pop
template<typename T>
bool pop(T closingSymbol){
    //check if the stack is empty
    if(head<T> == nullptr){
        std::cout << "Stack Underflow" << std::endl;
        return false;
    }

    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
    T topChar = temp->data;
    //Check if the Top Character matches the closing symbol
    bool match = (topChar == '(' && closingSymbol == ')') ||
                 (topChar == '[' && closingSymbol == ']') ||
                 (topChar == '{' && closingSymbol == '}');
    //If the symbols do not match, report an error and return false
    if (!match) {
        std::cout << "ERROR: expected matching symbol for: " << topChar << ", but found: " << closingSymbol << ".\n";
        return false;
    }
    //move the head to next node
    head<T> = head<T> -> next;
    //free the removed node
    delete temp;
    //return the removed node
    return true;

}
```

```cpp

//isEmpty
template<typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

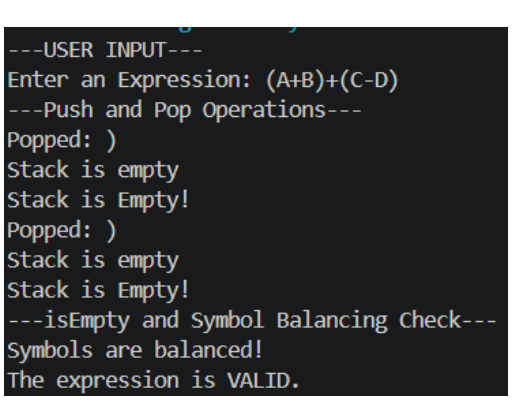
//top
template<typename T>
void Top(){
    // error catching
    if (head<T> == nullptr) {
        std::cout << "Stack is empty" << std::endl;
        return;
    }
    // display the top node
    std::cout << "The Current Top of the stack is: " << head<T>->data << std::endl;
}

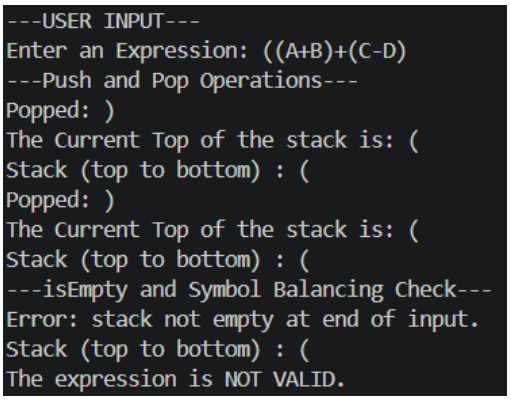
//displayAll
template<typename T>
void displayAll(){
    if (isEmpty<T>()){
        std::cout<< "Stack is Empty! \n";
        return;
    }

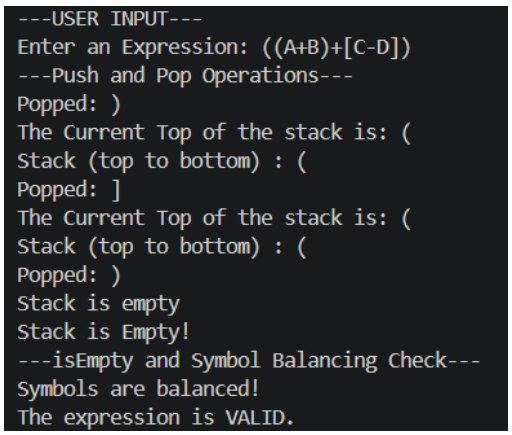
    std::cout << "Stack (top to bottom) : ";
    Node<T>* current = head<T>;
    while (current != nullptr) {
        std::cout << current -> data;
        if (current -> next != nullptr) std::cout << " -> ";
        current = current -> next;
    }
    std::cout << std::endl;
}

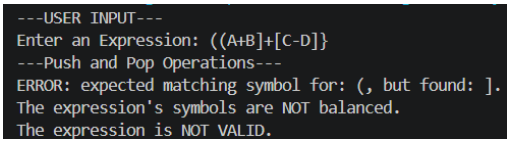

#endif
```

HEADER FILE CODE

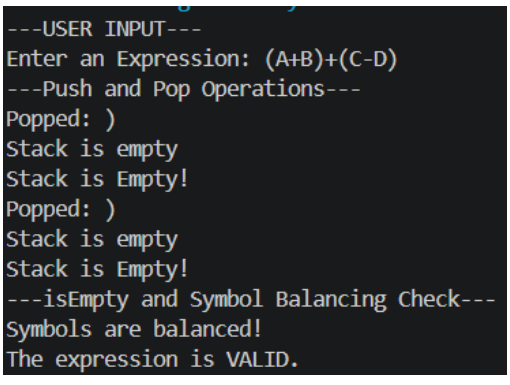


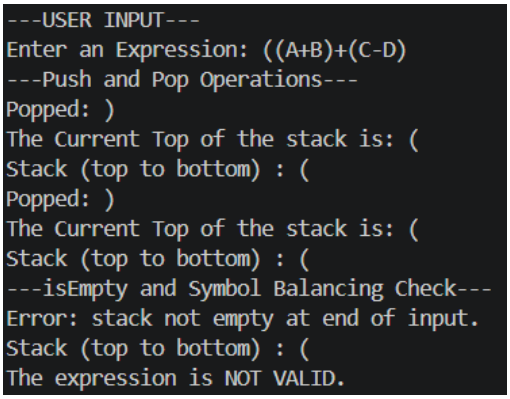


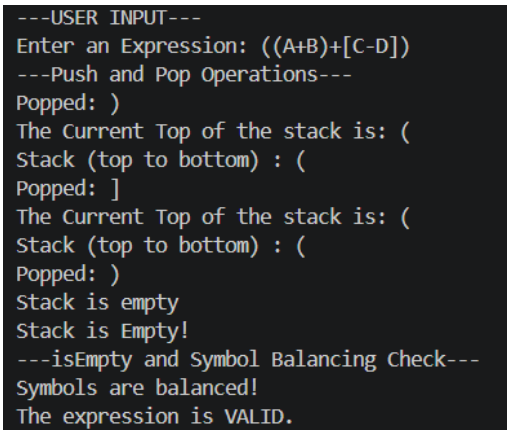
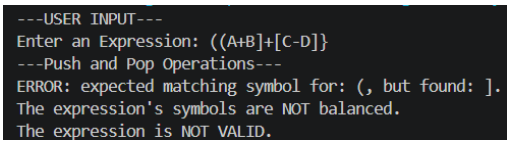




OUTPUT

Expressions	Valid (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		In this output, First, you must input the expression which is **(A + B) + (C - D)** in the **USER INPUT**. Second, the **PUSH AND POP OPERATIONS** executes that **each closing parentheses triggered a pop operation**, and **the stack became empty after each pop, confirming that every opening symbol had a matching**

			closing one which is **Valid** so the program **displays "Stack is Empty" twice for the Top<char>() and displayAll<char>()**. Third, the **isEmpty and Symbol Balancing Check** concluded with **"Symbols are balanced!" and "The expression is VALID."**.
((A+B)+(C-D)	N		In this output, First, you must input the expression which is **((A + B) + (C - D)** in the **USER INPUT**. Second, the **PUSH AND POP OPERATIONS** concluded that **one open parenthesis remained in the stack at the end of the input** which **displays "Error: stack not empty at the end of input" that confirms the symbol imbalance**. The **subsequent display of the stack contents showing only 'C' and pinpoints the leftover symbol causing the error**. Finally, the program concludes in the **isEmpty and Symbol Balancing Check** that **"The expression is NOT VALID"** because of its **improper symbol balancing**.

((A+B)+[C-D])	Y		In this output, First, you must input the expression which is **((A + B) + [C - D])** in the **USER INPUT**. Second, the **PUSH AND POP OPERATIONS** executes that **each parentheses and brackets triggered a pop operation**, and the **stack became empty after each pop, confirming that every opening symbol had a matching closing one which is Valid** so the program **displays "Stack is Empty" twice for the Top<char>() and displayAll<char>()**. Third, the **isEmpty and Symbol Balancing Check** concluded with **"Symbols are balanced!" and "The expression is VALID." which handled the mixed symbol expression types efficiently and accurately**.
((A+B]+[C-D]}	N		In this output, First, you must input the expression which is **((A + B] + [C - D]}** in the **USER INPUT**. Second, the **PUSH AND POP OPERATIONS** concludes that **it expected a closing parenthesis to match the opening parenthesis, but instead encountered a closing bracket** which the program **concluded that the expression's**

			symbols are NOT BALANCED, and therefore NOT VALID.

Conclusion and Lessons Learned	In the Supplementary Activity, I've used the Linked List Stack Method which the program code demonstrates how the stack can be used in validating balanced symbols in an expression by modifying the Push and Pop Operations. By pushing the opening brackets and popping them when matching closing brackets appear, the algorithm of the code ensures proper pairing and detects mismatches or leftover symbols which it provides a clear output if the expression is Valid or Not by the condition in the Main Driver Code. Lastly, I've learned that Stacks also are used for proper data structure use, logical condition handling, and precise error handling in programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **NONE**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **NONE**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

COMPUTER ENGINEERING

QUEZON CITY



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.